Development and Evaluation of a U-Net Model for Multi-Class Semantic Segmentation Using the ADE20K Dataset

NAME: Ramya Thulluri **Student ID:** 23097466

GitHub link: https://github.com/thulluri-ramya/Image-Segmentation-Assignment-Research-Methods/tree/main

INTRODUCTION:

One of the major issues in computer vision is getting computers to perceive complicated scenes in the same way that humans do. Semantic image segmentation solves this problem by assigning each pixel in an image to a specific item category, resulting in a thorough description of the scene. Unlike typical picture classification or object detection approaches, this approach determines both the kind and specific location of objects. As a result, semantic segmentation has become a vital approach in fields such as autonomous driving, medical imaging, robotics, and intelligent surveillance, where precise scene interpretation is required.

The purpose of this project is to develop and evaluate a deep learning model capable of performing multi-class semantic segmentation on a subset of the ADE20K dataset. Unlike binary segmentation, which assigns pixels to either the item or the background, this research focuses on detecting four separate object categories: human, car, book, and aeroplane. A thorough pipeline was employed to extract the required classes, create segmentation masks, and prepare the data for model training. To predict pixel-level labels for the required classes, a U-Net model was built and trained in Google Colab using TensorFlow.

The proposed model's performance is quantitatively assessed using measures such as validation accuracy, Mean Intersection over Union (Mean IoU), Dice Score, and Pixel Accuracy, as well as qualitative comparisons between projected segmentation masks and ground-truth annotations. The findings are critically reviewed to establish the strengths and limitations of the approach used, as well as the impact of dataset characteristics, particularly class imbalance, on segmentation accuracy.

LITERATURE SURVEY:

Semantic image segmentation has advanced greatly since the introduction of deep learning techniques, particularly Convolutional Neural Networks. Long et al. (2015) proposed one of the first deep learning models developed specifically for semantic segmentation: the Fully Convolutional Network (FCN). Unlike traditional CNNs for image classification, FCNs employ convolutional layers rather than completely linked layers, allowing for end-to-end pixel-wise prediction. This architecture demonstrated that semantic segmentation could be carried out immediately, eliminating the requirement for manual feature extraction or separate classification methods. However, FCNs usually produced incorrect segmentation borders because repeated pooling operations reduced feature maps' spatial resolution.

To solve these issues, Ronneberger et al. (2015) introduced the U-Net architecture, which has since become one of the most widely used semantic segmentation models. U-Net transports fine-grained spatial information from the encoder to the decoder via an encoder-decoder structure connected by skip links. This method enables the network to preserve object boundaries while still learning high-level semantic features. U-Net is widely used in medical imaging, remote sensing, and general scene interpretation due to its simple architecture, efficient training approach, and remarkable performance

on small and medium-sized datasets. For these reasons, U-Net was selected as the primary segmentation model for this project.

Chen et al. (2018) introduced DeepLabv3+, which makes significant contributions to semantic segmentation. This architecture improves segmentation accuracy by combining atrous (dilated) convolution with an encoder-decoder structure, especially at complicated object borders. DeepLabv3+ surpasses earlier segmentation models on benchmark datasets; nonetheless, its increased computational complexity and training requirements make it more appropriate for large-scale datasets and advanced hardware.

DATA DESCRIPTION AND EDA:

The dataset employed in this study is a subset of the ADE20K semantic segmentation dataset, which is widely used in scene comprehension and computer vision research. The dataset available includes 350 training shots, 350 validation images, and 30 unlabelled test images. Each training and validation image has pixel-level annotations saved as COCO-style JSON files. Despite the dataset's 100 semantic object categories, just four target classes—person, automobile, book, and airplane—were chosen to match the assignment requirements.

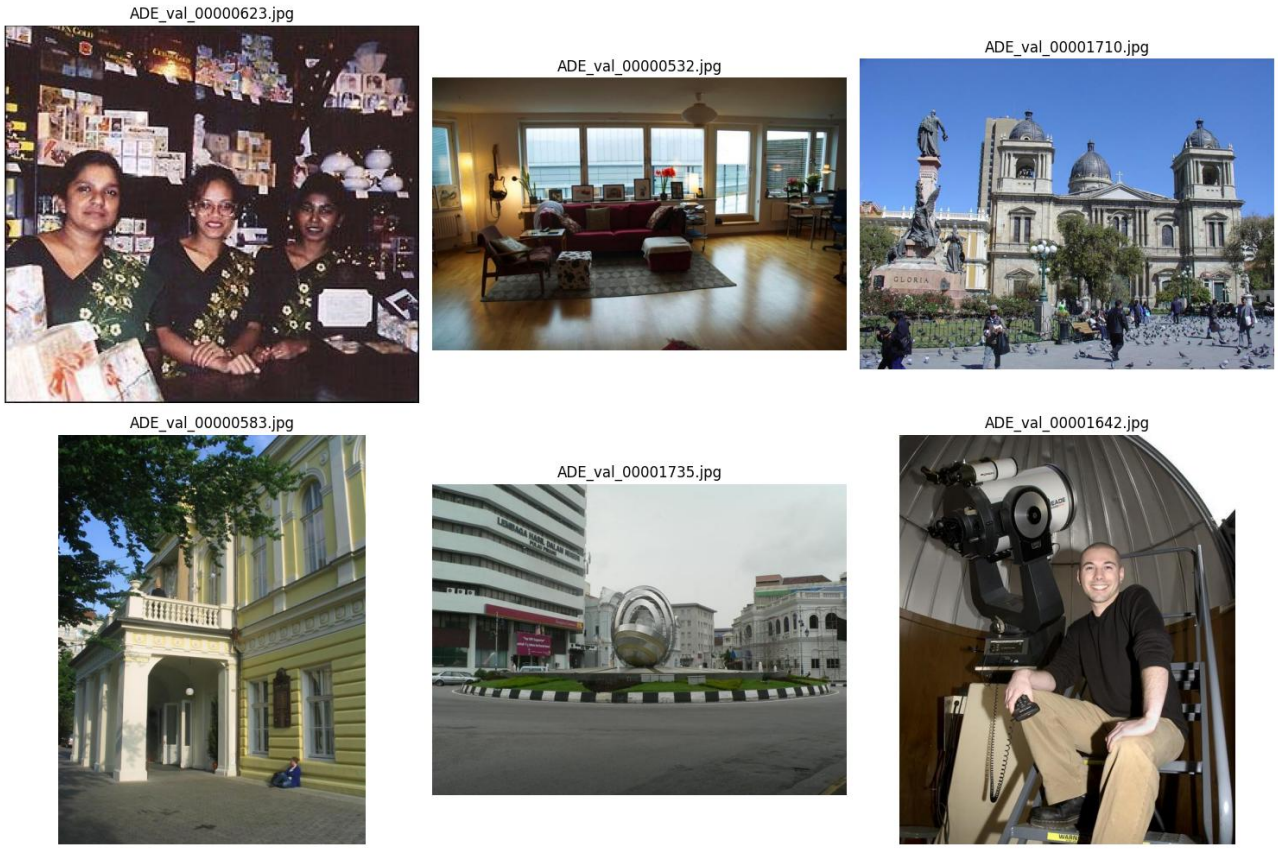

FIG 1: Random training photos from the ADE20K dataset, demonstrating the variety of interior and outdoor scenarios used in this study. (Source: Author's implementation based on the ADE20K dataset)

Prior to model building, exploratory data analysis (EDA) was used to evaluate the dataset's features and find any defects that could impact segmentation performance. The annotation files were first evaluated with the COCO API to ensure that the accessible item categories and category IDs were present. Random training photos and segmentation masks were analysed to ensure that the annotations were properly aligned with the source images. This visual examination also emphasised the collection's different interior and outdoor scenarios, object scales, angles, and lighting arrangements.

FIG 2: Distribution of the four Target Classes. The graphic demonstrates the significant disparity between common classes (person and car) and rare classes (book and aeroplane).

To prepare the data for training, a custom preprocessing pipeline was created that generated multi-class segmentation masks. The background was designated 0, while the selected target classes were labelled 1 (human), 2 (vehicle), 3 (book), and 4 (aeroplane). Images and masks were scaled to 256 × 256 pixels with nearest- neighbor interpolation to maintain class labels.

The filtered dataset showed a considerable class imbalance, according to the EDA. The human class occurred in 233 training images, followed by cars (148), books (51), and aeroplanes (5). Pixel-level analysis revealed that backdrop pixels made up roughly 93% of the sample, while aeroplane pixels made up less than 0.1% of all identified pixels. This disparity implied that the model would have many fewer samples from which to learn the rare classes, making them more difficult to segment effectively during training.

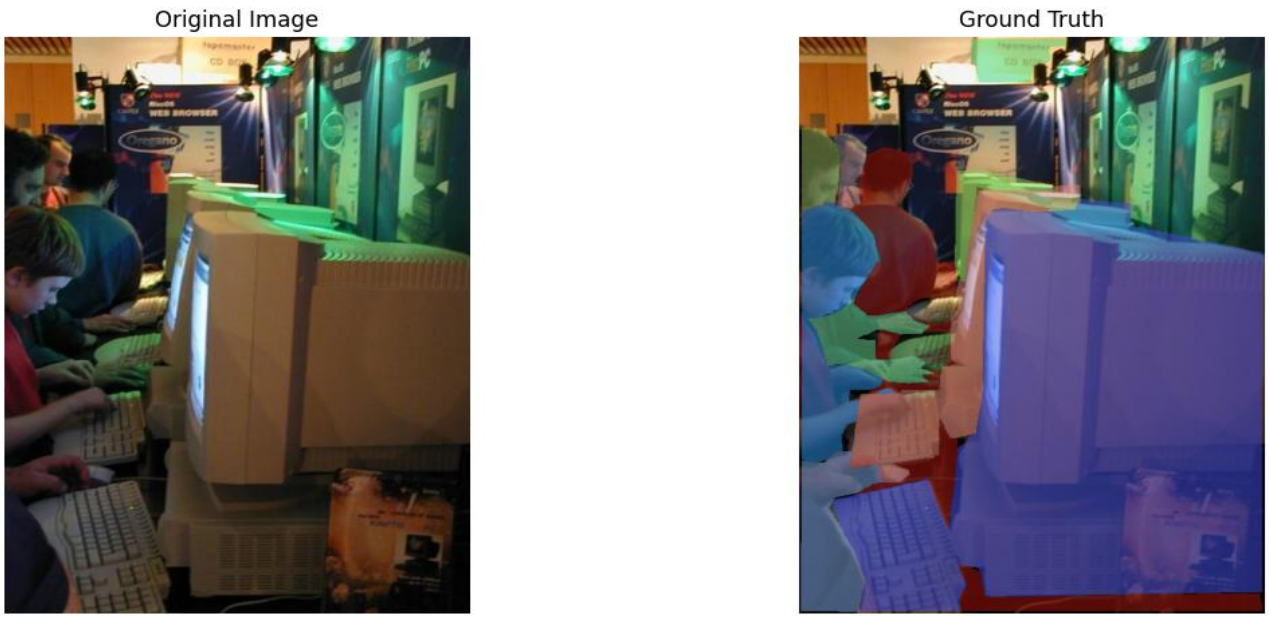

FIG 3: ground truth segmentation of the random image

METHODOLOGY:

The proposed semantic segmentation system was implemented using **Python** with the **TensorFlow/Keras** deep learning framework in **Google Colab**. The ADE20K dataset was processed using the COCO API to retrieve images and annotation information. Only images containing the four required classes (**person, car, book, and airplane**) were selected for training and validation. Multi-class segmentation masks were generated from the annotation files and resized to **256 × 256 pixels** together with the corresponding images. A **U-Net** model was developed to perform pixel-level classification and trained using the **Adam optimiser** with the **Sparse Categorical Cross-Entropy** loss function. To improve training efficiency and reduce overfitting, **Early Stopping** and **Model Checkpoint** callbacks were incorporated. The overall processing pipeline and model architecture are presented in Figures 4 and 5.

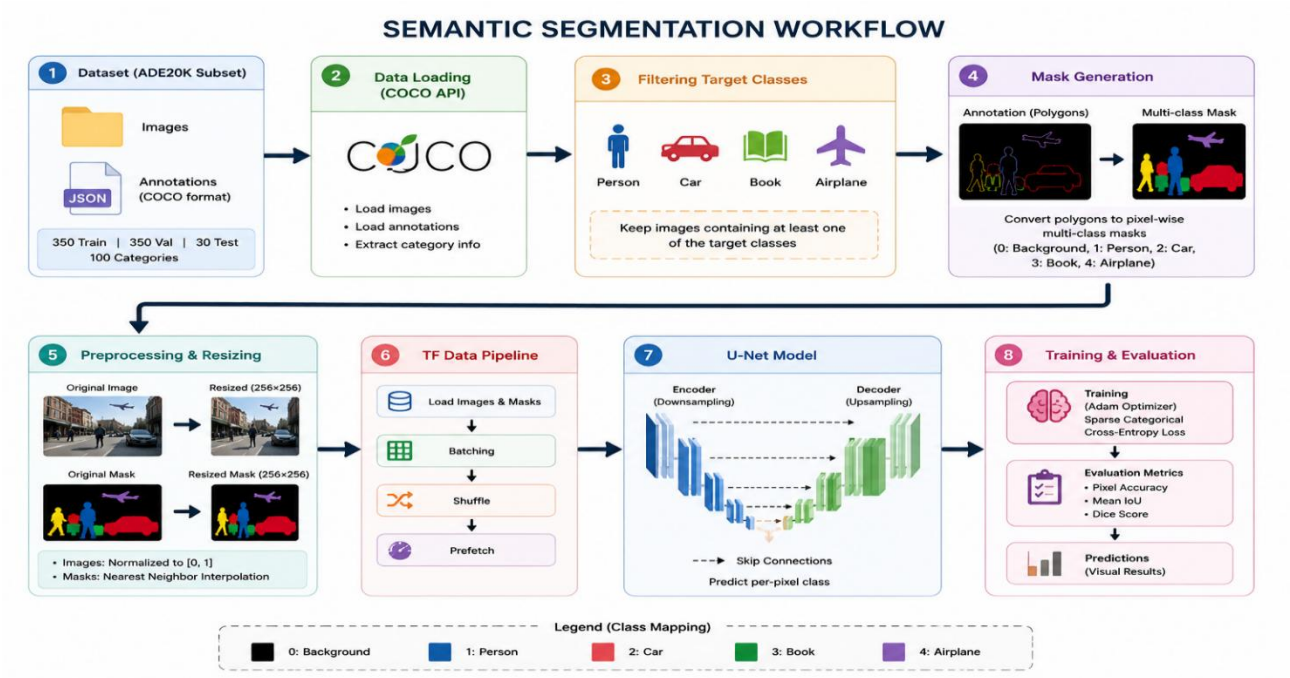

FIG 4: Figure shows the whole process of the proposed semantic segmentation system developed for this research.

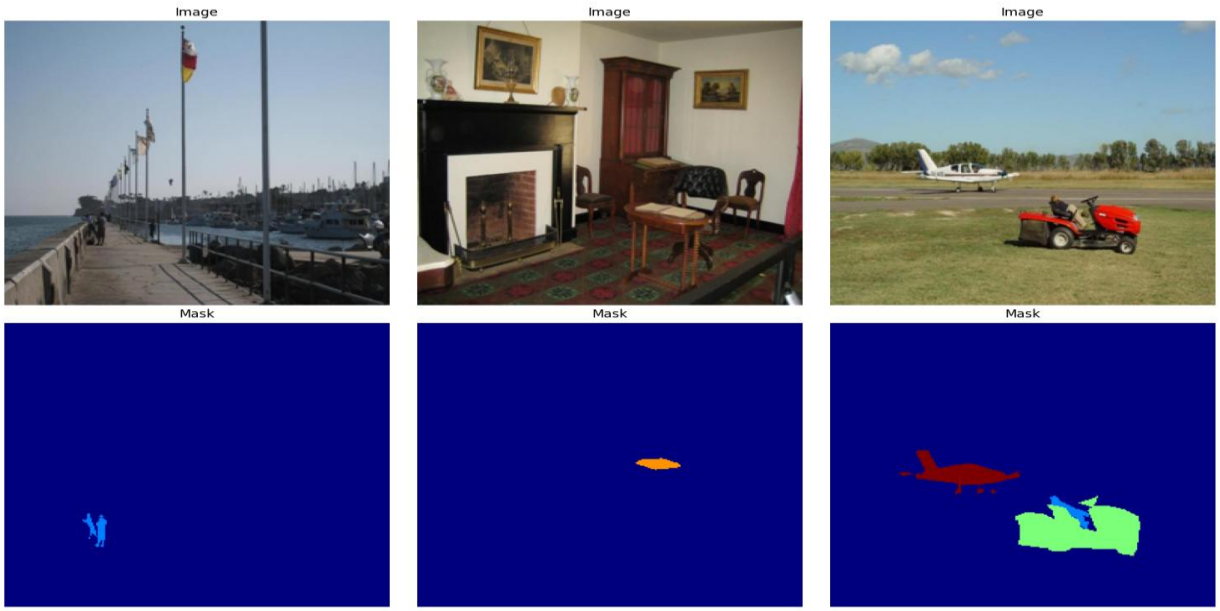

FIG 5: Data Pre – Processing in my m model

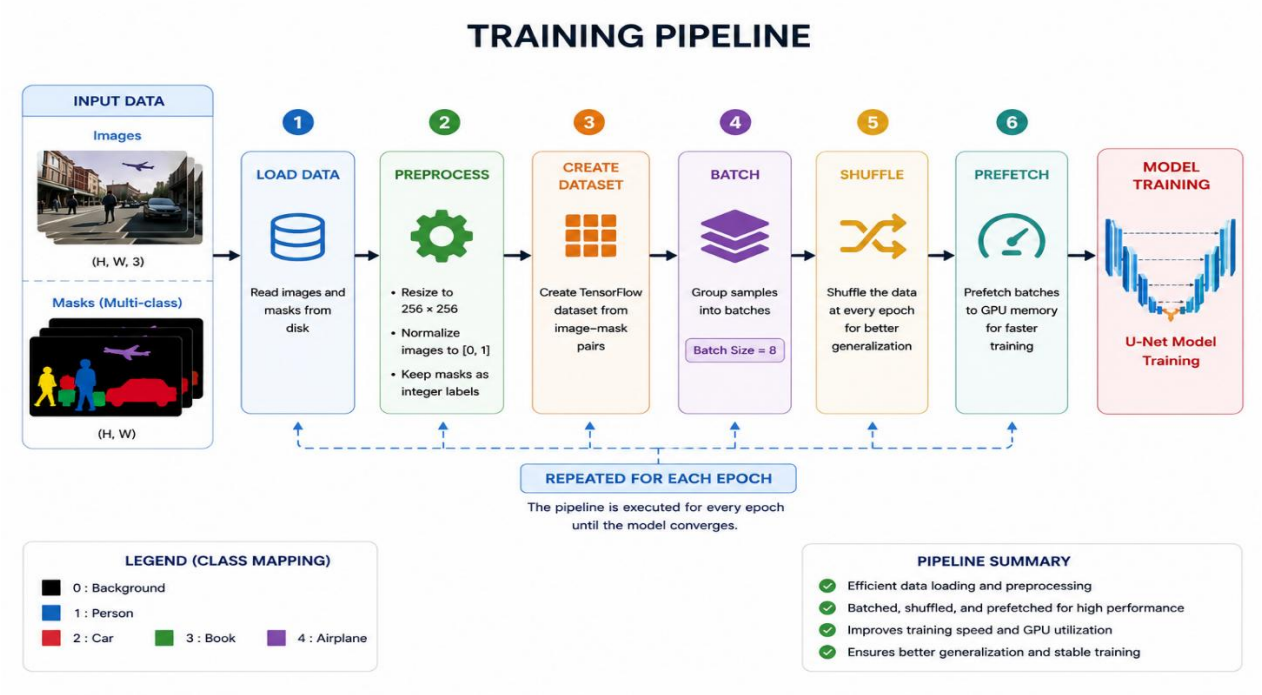

FIG 6: Training Pipeline flowchart

RESULTS AND DISCUSSIONS:

The suggested U-Net model was trained for 30 epochs utilising the Adam optimiser and the Sparse Categorical Cross-Entropy loss function. Early Stopping and Model Checkpoint were implemented during training to improve generalisation. The model had a training accuracy of 93.3%, a validation accuracy of 92.9%, and a validation loss of 0.264. The continuous reduction in validation loss, as well as the consistent accuracy values, suggest that the model effectively converged and learned meaningful features from training data.

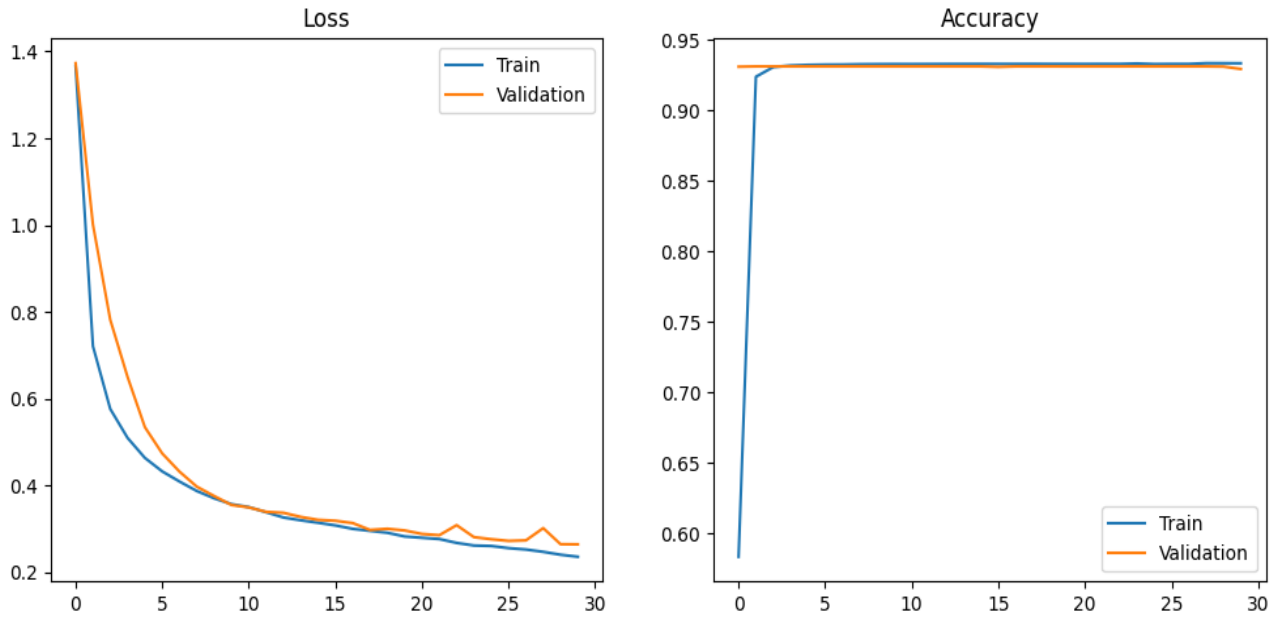

Fig 7: depicts the U-Net model's training and validation accuracy and loss curves across 30 training epochs.

Further study revealed that the model predicted only the background, person, and automobile classes, whereas the book and aircraft classes were rarely or never detected. This behaviour is consistent with the class distribution seen in the exploratory data analysis. The training dataset consisted of 233 photos of people, 148 of cars, 51 of books, and only 5 of aeroplanes. Furthermore, backdrop pixels accounted for about 93% of all recognised pixels, while aeroplane pixels accounted for less than 0.1%. As a result, the model received minimal training data for the odd classes, making it unable to build representative features for accurate segmentation.

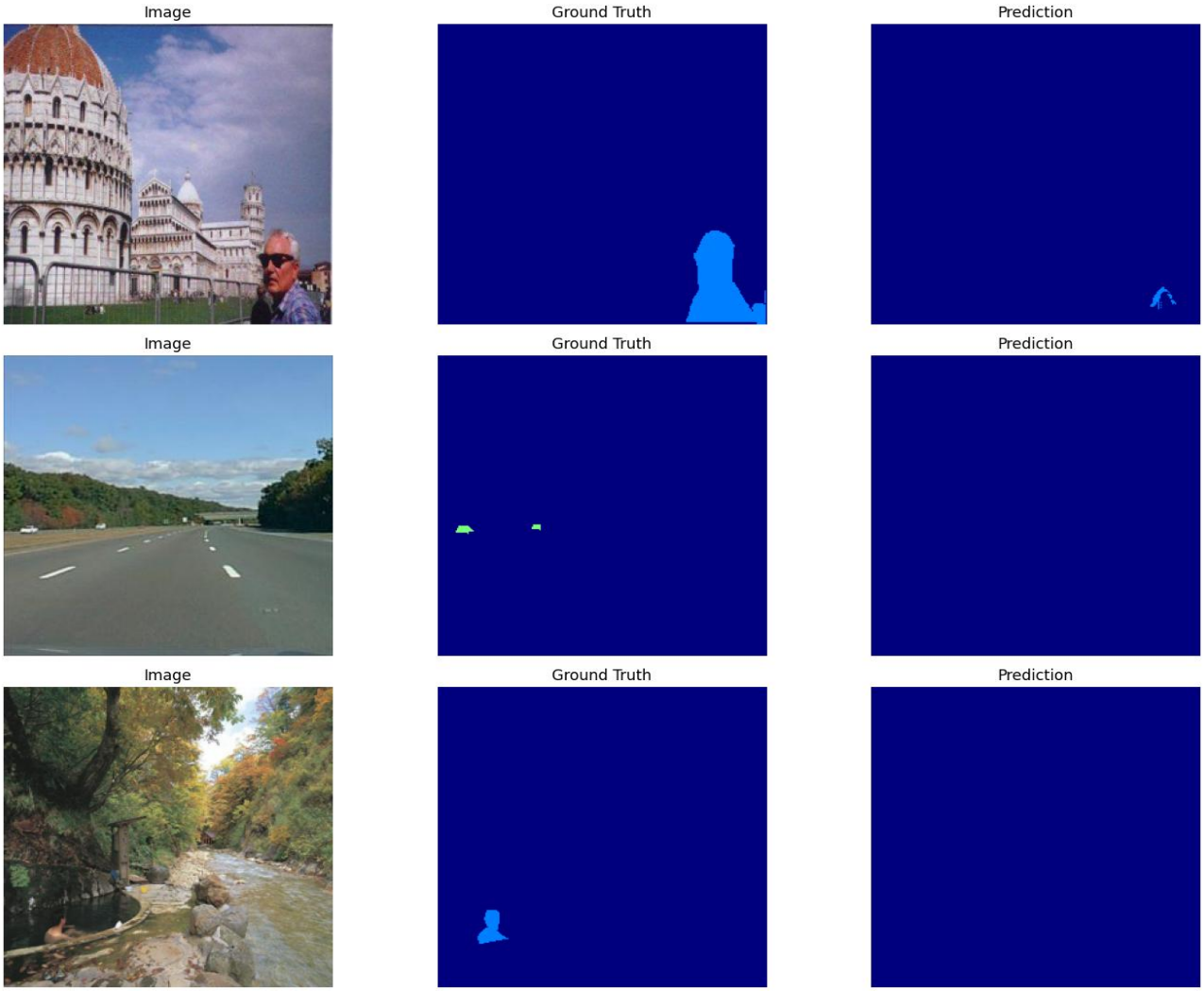

Fig 8: The original image, the ground-truth segmentation mask, and the segmentation that the trained U-Net model predicted are all compared. Although the model scored badly on rare classes, it correctly identified common object classes.

The results are consistent with previous research. Ronneberger, Fischer, and Brox (2015) demonstrated that U-Net performs well when sufficient annotated training data is available. Similarly, Chen et al. (2018) discovered that segmentation performance suffers for small or rare objects due to class imbalance and insufficient feature representation. The results of this experiment thus lend support to existing literature, suggesting that dataset features can have a considerable impact on semantic segmentation accuracy.

Several enhancements could be studied in future studies. Increase the number of training pictures with minority classes, utilise targeted data augmentation, or use imbalance-aware loss functions like Dice Loss or Focal Loss to improve segmentation performance. Furthermore, using a pretrained encoder like ResNet or MobileNet may improve feature extraction and recognition of small objects.

Table: Final Performance Metrix.

Metric	Value
Training Accuracy	**93.3%**
Validation Accuracy	**92.9%**
Validation Loss	**0.264**

REFERENCES:

1. Long, J., Shelhamer, E. and Darrell, T. (2015) *Fully Convolutional Networks for Semantic Segmentation*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3431–3440.
2. Ronneberger, O., Fischer, P. and Brox, T. (2015) *U-Net: Convolutional Networks for Biomedical Image Segmentation*. In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015)*. Lecture Notes in Computer Science, vol. 9351. Springer, Cham, pp. 234–241.
3. Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F. and Adam, H. (2018) *Encoder–Decoder with Atrous Separable Convolution for Semantic Image Segmentation*. Proceedings of the European Conference on Computer Vision (ECCV), pp. 801–818

APPENDIX:

"""research methods img ass.ipynb

Automatically generated by Colab.

Original file is located at

https://colab.research.google.com/drive/1Tg-VZa6qmoTXZV1he8Di-7rqHHBaKMQa

"""

```
import os

import json

import random

import zipfile

import warnings

warnings.filterwarnings("ignore")

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from PIL import Image
```

```
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from sklearn.model_selection import train_test_split
# COCO API
from pycocotools.coco import COCO
print("TensorFlow Version:", tf.__version__)
from google.colab import files
uploaded = files.upload()
# Extracting Dataset
zip_path = list(uploaded.keys())[0]
extract_path = "/content/ADE20K"
with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extract_path)
print("Dataset extracted successfully!")
for root, dirs, files in os.walk(extract_path):
    print(root)
import os
# Dataset Paths
TRAIN_PATH = "/content/ADE20K/train"
VAL_PATH = "/content/ADE20K/val"
TEST_PATH = "/content/ADE20K/test"
# Correcting JSON paths based on file system output
# The JSON files are directly under /content/ADE20K, not in subdirectories.
TRAIN_JSON = os.path.join("/content/ADE20K", "instances_train.json")
VAL_JSON = os.path.join("/content/ADE20K", "instances_val.json")
print("Train Path :", TRAIN_PATH)
print("Validation Path :", VAL_PATH)
print("Test Path :", TEST_PATH)
# Loading COCO Annotation Files
from pycocotools.coco import COCO
```

```
train_coco = COCO(TRAIN_JSON)
val_coco = COCO(VAL_JSON)
print("Training images:", len(train_coco.imgs))
print("Validation images:", len(val_coco.imgs))
# Verifyind  Categories
categories = train_coco.loadCats(train_coco.getCatIds())
for cat in categories:
    if cat["name"] in ["person", "car", "book", "airplane"]:
        print(cat)
# Class Mapping
CLASS_MAP = {
    4: 1,      # person
    9: 2,      # car
    40: 3,     # book
    59: 4      # airplane
}
CLASS_NAMES = {
    0: "Background",
    1: "Person",
    2: "Car",
    3: "Book",
    4: "Airplane"
}
NUM_CLASSES = 5
print(CLASS_MAP)
# Dataset Overview
print("========== DATASET OVERVIEW ==========\n")
print(f"Training Images   : {len(train_coco.imgs)}")
print(f"Validation Images : {len(val_coco.imgs)}")
print(f"Total Categories  : {len(train_coco.cats)}")
# Displaying Category Names
categories = train_coco.loadCats(train_coco.getCatIds())
```

```
category_names = [cat["name"] for cat in categories]
print(category_names)
# Target Classes
target_classes = ["person","car","book","airplane"]
print("Target Classes\n")
for cat in categories:
    if cat["name"] in target_classes:
        print(f"ID: {cat['id']} ---> {cat['name']}")
# Displaying Random Training Images
image_ids = list(train_coco.imgs.keys())
sample_ids = random.sample(image_ids,6)
plt.figure(figsize=(15,10))
for i,img_id in enumerate(sample_ids):
    img_info = train_coco.loadImgs(img_id)[0]
    img_path = os.path.join(TRAIN_PATH,img_info["file_name"])
    image = Image.open(img_path)
    plt.subplot(2,3,i+1)
    plt.imshow(image)
    plt.title(img_info["file_name"])
    plt.axis("off")
plt.tight_layout()
plt.show()
# Counting Images per Target Class
target_ids = [4,9,40,59]
class_counts = {}
for cat_id in target_ids:
    count = 0
    for img in train_coco.dataset["images"]:
        anns = train_coco.loadAnns(train_coco.getAnnIds(imgIds=img["id"]))
        found = False
        for ann in anns:
            if ann["category_id"] == cat_id:
```

```
                found = True
                break
        if found:
            count += 1
    class_counts[cat_id] = count
print(class_counts)
# Class Distribution
labels = ["Person","Car","Book","Airplane"]
values = list(class_counts.values())
plt.figure(figsize=(8,5))
plt.bar(labels,values)
plt.title("Distribution of Target Classes")
plt.xlabel("Classes")
plt.ylabel("Number of Images")
plt.show()
# Image Size Distribution
heights = []
widths = []
for img in train_coco.dataset["images"]:
    heights.append(img["height"])
    widths.append(img["width"])
print("Minimum Height :",min(heights))
print("Maximum Height :",max(heights))
print("Minimum Width :",min(widths))
print("Maximum Width :",max(widths))
# Visualizing Ground Truth Masks
sample_id = random.choice(list(train_coco.imgs.keys()))
img_info = train_coco.loadImgs(sample_id)[0]
img_path = os.path.join(TRAIN_PATH,img_info["file_name"])
image = Image.open(img_path)
anns = train_coco.loadAnns(train_coco.getAnnIds(imgIds=sample_id))
plt.figure(figsize=(14,6))
```

```
plt.subplot(1,2,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1,2,2)
plt.imshow(image)
train_coco.showAnns(anns)
plt.title("Ground Truth")
plt.axis("off")
plt.show()
# Dataset Summary
print("========== DATA SUMMARY ==========\n")
print("Training Images :",len(train_coco.imgs))
print("Validation Images :",len(val_coco.imgs))
print("Number of Categories :",len(train_coco.cats))
print("\nTarget Categories")
for cat in categories:
    if cat["id"] in [4,9,40,59]:
        print(cat["name"])
# Dataset Filtering & Mask Generation
IMG_SIZE = 256
TARGET_CLASSES = {
    4: 1,      # Person
    9: 2,      # Car
    40: 3,     # Book
    59: 4      # Airplane
}
NUM_CLASSES = 5
# Finding Training Images with Target Objects
train_image_ids = []
for img in train_coco.dataset["images"]:
    anns = train_coco.loadAnns(
```

```
        train_coco.getAnnIds(imgIds=img["id"])
    )
    if any(ann["category_id"] in TARGET_CLASSES for ann in anns):
        train_image_ids.append(img["id"])
print("Training Images Found:", len(train_image_ids))
print(train_image_ids[:10])
# Finding Validation Images
val_image_ids = []
for img in val_coco.dataset["images"]:
    anns = val_coco.loadAnns(
        val_coco.getAnnIds(imgIds=img["id"])
    )
    if any(ann["category_id"] in TARGET_CLASSES for ann in anns):
        val_image_ids.append(img["id"])
print("Validation Images Found:", len(val_image_ids))
# Creating Multi-Class Segmentation Mask
def create_mask(coco, img_id):
    img_info = coco.loadImgs(img_id)[0]
    h = img_info["height"]
    w = img_info["width"]
    mask = np.zeros((h, w), dtype=np.uint8)
    ann_ids = coco.getAnnIds(imgIds=img_id)
    anns = coco.loadAnns(ann_ids)
    for ann in anns:
        category = ann["category_id"]
        if category not in TARGET_CLASSES:
            continue
        object_mask = coco.annToMask(ann)
        mask[object_mask == 1] = TARGET_CLASSES[category]
    return mask
# Visualising One Mask
sample_id = random.choice(train_image_ids)
```

```
mask = create_mask(train_coco, sample_id)
print("Unique labels:", np.unique(mask))
plt.figure(figsize=(6,6))
plt.imshow(mask, cmap="jet", vmin=0, vmax=4)
plt.colorbar()
plt.title("Ground Truth Mask")
plt.axis("off")
plt.show()
# Loading Training Images and Masks
train_images = []
train_masks = []
for img_id in train_image_ids:
    info = train_coco.loadImgs(img_id)[0]
    img_path = os.path.join(TRAIN_PATH, info["file_name"])
    image = Image.open(img_path).convert("RGB")
    image = image.resize((IMG_SIZE, IMG_SIZE))
    image = np.array(image, dtype=np.float32) / 255.0
    mask = create_mask(train_coco, img_id)
    mask = Image.fromarray(mask)
    mask = mask.resize((IMG_SIZE, IMG_SIZE), Image.NEAREST)
    mask = np.array(mask, dtype=np.uint8)
    train_images.append(image)
    train_masks.append(mask)
train_images = np.array(train_images, dtype=np.float32)
train_masks = np.array(train_masks, dtype=np.uint8)
print(train_images.shape)
print(train_masks.shape)
# Loading Validation Images and Masks
val_images = []
val_masks = []
for img_id in val_image_ids:
    info = val_coco.loadImgs(img_id)[0]
```

```
    img_path = os.path.join(VAL_PATH, info["file_name"])
    image = Image.open(img_path).convert("RGB")
    image = image.resize((IMG_SIZE, IMG_SIZE))
    image = np.array(image, dtype=np.float32) / 255.0
    mask = create_mask(val_coco, img_id)
    mask = Image.fromarray(mask)
    mask = mask.resize((IMG_SIZE, IMG_SIZE), Image.NEAREST)
    mask = np.array(mask, dtype=np.uint8)
    val_images.append(image)
    val_masks.append(mask)
val_images = np.array(val_images, dtype=np.float32)
val_masks = np.array(val_masks, dtype=np.uint8)
print(val_images.shape)
print(val_masks.shape)
# Verifying Labels
print("Training Labels :", np.unique(train_masks))
print("Validation Labels:", np.unique(val_masks))
# Displaying Samples
plt.figure(figsize=(15,10))
for i in range(3):
    plt.subplot(2,3,i+1)
    plt.imshow(train_images[i])
    plt.title("Image")
    plt.axis("off")
    plt.subplot(2,3,i+4)
    plt.imshow(train_masks[i], cmap="jet", vmin=0, vmax=4)
    plt.title("Mask")
    plt.axis("off")
plt.tight_layout()
plt.show()
#TensorFlow Dataset Pipeline
print("Training Images Shape :", train_images.shape)
```

```
print("Training Masks Shape  :", train_masks.shape)
print("Validation Images Shape :", val_images.shape)
print("Validation Masks Shape  :", val_masks.shape)
# Converting Masks
train_masks = train_masks.astype(np.uint8)
val_masks = val_masks.astype(np.uint8)
print(train_masks.dtype)
print(val_masks.dtype)
# Hyperparameters
BATCH_SIZE = 8
AUTOTUNE = tf.data.AUTOTUNE
print("Batch Size:", BATCH_SIZE)
# Building Training Dataset
train_dataset = tf.data.Dataset.from_tensor_slices(
    (train_images, train_masks)
)
val_dataset = tf.data.Dataset.from_tensor_slices(
    (val_images, val_masks)
)
# Dataset Preprocessing
def preprocess(image, mask):
    image = tf.cast(image, tf.float32)
    mask = tf.cast(mask, tf.uint8)
    return image, mask
# Applying Preprocessing
train_dataset = train_dataset.map(
    preprocess,
    num_parallel_calls=AUTOTUNE
)
val_dataset = val_dataset.map(
    preprocess,
    num_parallel_calls=AUTOTUNE
```

```
)
# Optimizing Dataset Pipeline
train_dataset = (
    train_dataset
    .shuffle(500)
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)
val_dataset = (
    val_dataset
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)
# Verifying Dataset
for images, masks in train_dataset.take(1):
    print("Images :", images.shape)
    print("Masks  :", masks.shape)
# Visualizing Training Batch
for images, masks in train_dataset.take(1):
    plt.figure(figsize=(15,10))
    for i in range(3):
        plt.subplot(2,3,i+1)
        plt.imshow(images[i])
        plt.title("Image")
        plt.axis("off")
        plt.subplot(2,3,i+4)
        plt.imshow(masks[i], cmap="jet", vmin=0, vmax=4)
        plt.title("Mask")
        plt.axis("off")
    plt.tight_layout()
    plt.show()
#U-Net Model
```

```
from tensorflow.keras.layers import (
    Input,
    Conv2D,
    MaxPooling2D,
    Conv2DTranspose,
    concatenate,
    BatchNormalization,
    Activation,
    Dropout
)
def conv_block(inputs, filters):
    x = Conv2D(filters, 3, padding="same")(inputs)
    x = BatchNormalization()(x)
    x = Activation("relu")(x)
    x = Conv2D(filters, 3, padding="same")(x)
    x = BatchNormalization()(x)
    x = Activation("relu")(x)
    return x
def encoder_block(inputs, filters):
    x = conv_block(inputs, filters)
    p = MaxPooling2D((2,2))(x)
    p = Dropout(0.2)(p)
    return x, p
def decoder_block(inputs, skip, filters):
    x = Conv2DTranspose(filters, 2, strides=2, padding="same")(inputs)
    x = concatenate([x, skip])
    x = Dropout(0.2)(x)
    x = conv_block(x, filters)
    return x
IMG_SIZE = 256
NUM_CLASSES = 5
```

```
inputs = Input((IMG_SIZE, IMG_SIZE, 3))

# Encoder
s1, p1 = encoder_block(inputs, 64)
s2, p2 = encoder_block(p1, 128)
s3, p3 = encoder_block(p2, 256)
s4, p4 = encoder_block(p3, 512)

# Bridge
b1 = conv_block(p4, 1024)

# Decoder
d1 = decoder_block(b1, s4, 512)
d2 = decoder_block(d1, s3, 256)
d3 = decoder_block(d2, s2, 128)
d4 = decoder_block(d3, s1, 64)

outputs = Conv2D(
    NUM_CLASSES,
    1,
    activation="softmax"
)(d4)

model = Model(inputs, outputs)

model.summary()

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=[
        tf.keras.metrics.SparseCategoricalAccuracy(name="accuracy")
    ]
)

# Callbacks

checkpoint = ModelCheckpoint(

    "best_unet.keras",

    monitor="val_loss",

    save_best_only=True,

    verbose=1

)
```

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True,
    verbose=1
)

# Model trainig
EPOCHS = 30

history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=EPOCHS,
    callbacks=[
        checkpoint,
        early_stop
    ]
)

#Training Curves
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

plt.plot(history.history["loss"], label="Train")
plt.plot(history.history["val_loss"], label="Validation")

plt.title("Loss")

plt.legend()

plt.subplot(1,2,2)

plt.plot(history.history["accuracy"], label="Train")
plt.plot(history.history["val_accuracy"], label="Validation")
```

```
plt.title("Accuracy")

plt.legend()

plt.show()

model.save("ADE20K_UNet_Model.keras")

print("Model saved successfully.")

#Evaluation of the Model

results = model.evaluate(val_dataset)

print("\nValidation Results")
print("----------------------")
print(f"Validation Loss     : {results[0]:.4f}")
print(f"Validation Accuracy : {results[1]:.4f}")

# Predicting Validation Images

predictions = model.predict(val_images)

pred_masks = np.argmax(predictions, axis=-1)

print(pred_masks.shape)

# Pixel Accuracy

pixel_accuracy = np.mean(pred_masks == val_masks)

print(f"Pixel Accuracy: {pixel_accuracy:.4f}")

# Mean IoU

iou_metric = tf.keras.metrics.MeanIoU(num_classes=NUM_CLASSES)

iou_metric.update_state(val_masks, pred_masks)

print("Mean IoU:", iou_metric.result().numpy())

# Dice Score

def dice_score(y_true, y_pred):

    scores = []

    for cls in range(1, NUM_CLASSES):
```

```
        true_cls = (y_true == cls)
        pred_cls = (y_pred == cls)

        intersection = np.logical_and(true_cls, pred_cls).sum()

        denominator = true_cls.sum() + pred_cls.sum()

        if denominator == 0:
            continue

        dice = (2.0 * intersection) / denominator

        scores.append(dice)

    return np.mean(scores)

dice = dice_score(val_masks, pred_masks)

print(f"Dice Score: {dice:.4f}")

# Comparison of Ground Truth vs Prediction

plt.figure(figsize=(15,12))

samples = random.sample(range(len(val_images)),3)

for i, idx in enumerate(samples):

    plt.subplot(3,3,3*i+1)
    plt.imshow(val_images[idx])
    plt.title("Image")
    plt.axis("off")

    plt.subplot(3,3,3*i+2)
    plt.imshow(val_masks[idx], cmap="jet", vmin=0, vmax=4)
    plt.title("Ground Truth")
    plt.axis("off")

    plt.subplot(3,3,3*i+3)
    plt.imshow(pred_masks[idx], cmap="jet", vmin=0, vmax=4)
    plt.title("Prediction")
    plt.axis("off")

plt.tight_layout()
plt.show()

#Test Images
```

```
test_files = sorted([
    f for f in os.listdir(TEST_PATH)
    if f.endswith(".jpg")
])

print("Total Test Images:", len(test_files))

# Prediction of  Test Images

plt.figure(figsize=(15,30))

for i, file in enumerate(test_files[:9]):

    img = Image.open(os.path.join(TEST_PATH,file)).convert("RGB")

    img = img.resize((IMG_SIZE,IMG_SIZE))

    img_array = np.array(img,dtype=np.float32)/255.0

    prediction = model.predict(
        np.expand_dims(img_array,0),
        verbose=0
    )

    prediction = np.argmax(prediction[0],axis=-1)

    plt.subplot(9,2,2*i+1)
    plt.imshow(img)
    plt.title(file)
    plt.axis("off")

    plt.subplot(9,2,2*i+2)
    plt.imshow(prediction,cmap="jet",vmin=0,vmax=4)
    plt.title("Prediction")
    plt.axis("off")

plt.tight_layout()
plt.show()

# Saving the Predicted Masks

OUTPUT_DIR = "Predicted_Masks"

os.makedirs(OUTPUT_DIR, exist_ok=True)

for file in test_files:
```

```
    img = Image.open(os.path.join(TEST_PATH,file)).convert("RGB")

    img = img.resize((IMG_SIZE,IMG_SIZE))

    img_array = np.array(img,dtype=np.float32)/255.0

    prediction = model.predict(
        np.expand_dims(img_array,0),
        verbose=0
    )

    prediction = np.argmax(prediction[0],axis=-1)

    Image.fromarray(prediction.astype(np.uint8)).save(
        os.path.join(OUTPUT_DIR,file.replace(".jpg","_mask.png"))
    )

print("All predicted masks have been saved.")

print(np.unique(pred_masks))

classes = {
    0: "Background",
    1: "Person",
    2: "Car",
    3: "Book",
    4: "Airplane"
}

for c in range(5):
    pixels = np.sum(train_masks == c)
    print(f"{classes[c]:12}: {pixels}")

import numpy as np

pixel_counts = np.array([
    21399309,   # Background
    915021,     # Person
    497524,     # Car
    105835,     # Book
    19911       # Airplane
], dtype=np.float32)

weights = pixel_counts.sum() / (len(pixel_counts) * pixel_counts)

weights = weights / weights.min()

print(weights)
```

```
print("Training Images:", train_images.shape[0])
print("Validation Images:", val_images.shape[0])

image_counts = {1: 0, 2: 0, 3: 0, 4: 0}

for mask in train_masks:
    labels = np.unique(mask)
    for cls in [1, 2, 3, 4]:
        if cls in labels:
            image_counts[cls] += 1

print(image_counts)

print("Total training images:", len(train_coco.imgs))

airplane_img_ids = train_coco.getImgIds(catIds=[59])
print("Airplane images from COCO:", len(airplane_img_ids))
print(airplane_img_ids)

# ============================================================
# Data Augmentation
# ============================================================

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.08),
    tf.keras.layers.RandomContrast(0.10)
])

def preprocess(image, mask):

    image = tf.cast(image, tf.float32)
    mask = tf.cast(mask, tf.uint8)

    image = data_augmentation(image)

    return image, mask

optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4)

lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=1e-4,
    decay_steps=1000,
    decay_rate=0.9
)

optimizer = tf.keras.optimizers.Adam(lr_schedule)
```

```
model.compile(
    optimizer=optimizer,
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=[
        tf.keras.metrics.SparseCategoricalAccuracy(name="accuracy")
    ]
)

model = tf.keras.models.load_model("best_unet.keras")

history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=10,
    callbacks=[checkpoint, early_stop]
)

predictions = model.predict(val_images)

pred_masks = np.argmax(predictions, axis=-1)

print(np.unique(pred_masks))
```